Binary Search Trees

CSE 4303: Data Structures
Lecture 6

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

November 24, 2025

What is a Binary Search Tree?

- **Definition:** A Binary Search Tree (BST), also known as an *Ordered Binary Tree*, is a variant of a binary tree where nodes are arranged in a specific order.
- **The BST Property:**
  - **Left Subtree:** All nodes in the left sub-tree have a value **less than** the root node.
  - **Right Subtree:** All nodes in the right sub-tree have a value **greater than** (or equal to) the root node.
  - **Recursive Nature:** This rule applies to every sub-tree within the tree.

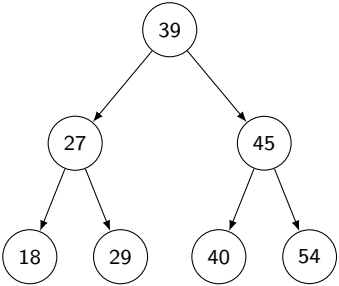
Note on Duplicates

Depending on the implementation, a BST may or may not contain duplicate values.

Visualizing a BST

Analyzing Figure 10.1:

- **Root Node:** 39
- **Left Subtree:** All values (e.g., 27, 18, 29) are < 39.
- **Right Subtree:** All values (e.g., 45, 54, 60) are > 39.
- **Sub-tree Check:** Node 27 is the root of the left sub-tree.
  - Left of 27: $\{9, 10, 18 \ldots\} < 27$
  - Right of 27: $\{28, 29 \ldots\} > 27$

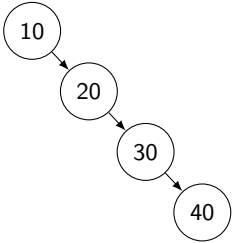


Efficiency of Binary Search Trees

- **Search Efficiency:**
  - The ordered nature allows us to skip entire sub-trees during a search.
  - **Average Case:** $O(\log_2 n)$. At every step, we eliminate half the search space.
- **Comparison with Linear Structures:**
  - **vs. Sorted Arrays:** Arrays offer $O(\log n)$ search but expensive insertion/deletion.
  - **vs. Linked Lists:** Lists offer easy insertion but slow $O(n)$ search.
  - **BST Sweet Spot:** Efficient for data that changes rapidly (frequent insertions and deletions).

The Worst Case: Skewed Trees

- **Problem:** If data is inserted in sorted order (e.g., 10, 20, 30), the tree becomes a linear chain.
- **Skewed Tree:** Effectively a Linked List.
- **Worst Case Complexity:** $O(n)$ for search, insertion, and deletion.



Right Skewed Tree

Beyond Basic BST

To handle the worst-case scenarios (like skewed trees), we use specialized "Efficient Binary Trees".

- 👉 **AVL Trees:** Self-balancing binary search trees that maintain $O(\log n)$ height.
- 👉 **Threaded Binary Trees:** Make use of NULL pointers to speed up traversal.
- 👉 **Red-Black Trees:** Another form of self-balancing tree used in system libraries.
- 👉 **Splay Trees:** Adjusts the tree so frequently accessed elements are near the top.

These structures extend the BST concept to guarantee efficiency.

BST Operation: Searching

- **Logic:**
  - The search process begins at the **Root**.
  - Compare the `SearchValue` with the `CurrentNode`.
- **Decision Making:**
  - If `SearchValue` == `CurrentNode`: Return **Found**.
  - If `SearchValue` < `CurrentNode`: Move to the **Left Child**.
  - If `SearchValue` > `CurrentNode`: Move to the **Right Child**.

Algorithm Snapshot

```
Node* search(Node* root, val){
  if(root==NULL or root->data==val)
      return root
  if(val < root->val)
      return search(root->left, val)
  else
      return search(root->right,val)
}
```

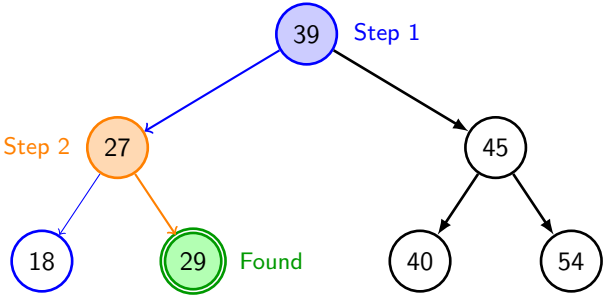
- **Termination:** Repeat until the value is found or the sub-tree is NULL (not found).

Visualizing Search: Step-by-Step

Example: Searching for the value **29**.

Trace:

1. **Step 1:** Start at Root **39**.
   $29 < 39 \rightarrow$ Go **Left**.
2. **Step 2:** Current is **27**.
   $29 > 27 \rightarrow$ Go **Right**.
3. **Step 3:** Current is **29**.
   $29 == 29 \rightarrow$ **Found!**



Legend: • Compare → • Compare → • Match

BST Operation: Insertion Logic

The Goal

Add a new node with value V while maintaining the property: `Left` < `Root` $\leq$ `Right`.

Algorithm Steps:

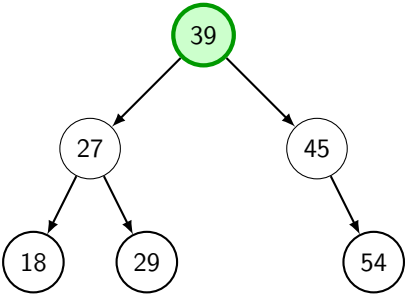
1. **Search:** Start at the Root. Compare V with the current node's data.
2. **Branching:**
   - If $V <$ Current, move to the **Left child**.
   - If $V >$ Current, move to the **Right child**.
3. **Placement:** Repeat Step 2 until you attempt to move to a `NULL` (empty) position.
4. **Attach:** Create a new node and attach it at that `NULL` position.

*New nodes are always inserted as **leaf nodes** in a basic BST.*

C++ Implementation: BST Insertion

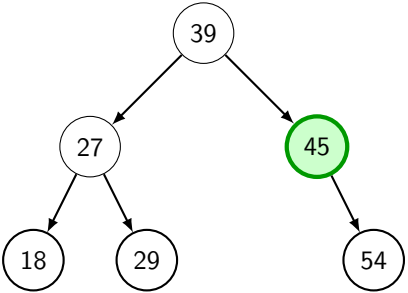
```
Node* insert(Node* root, int val) {
    // 1. If tree is empty, return a new node
    if (root == nullptr) {
        return new Node(val);
    }

    // 2. Otherwise, recur down the tree
    if (val < root->data) {
        root->left = insert(root->left, val);
    } else if (val > root->data) {
        root->right = insert(root->right, val);
    }

    // 3. Return the (unchanged) node pointer
    return root;
}
```

Visualizing Insertion: Adding 42



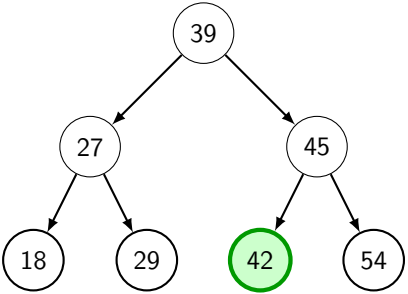

```
    // val = 42, root->data = 39
    } else if (val > root->data) {
        root->right = insert(root->right, val);
    }
```

Visualizing Insertion: Adding 42




```
// val = 42, root->data = 45
if (val < root->data) {
    root->left = insert(root->left, val);
```

Visualizing Insertion: Adding 42




```
// val = 42, root = nullptr on left of 45
// 1. If tree is empty, return a new node
if (root == nullptr) {
    return new Node(val);
}
```

BST Operation: Deletion Overview

Deleting a node requires re-linking the tree to maintain the BST property. The complexity depends on the number of children the target node has:

1. **Case 1: Node is a Leaf**
   The simplest case. Just remove the node.
2. **Case 2: Node has One Child**
   The child "jumps up" to take the place of the deleted parent.
3. **Case 3: Node has Two Children**
   The most complex. We must find a replacement node to maintain the order.

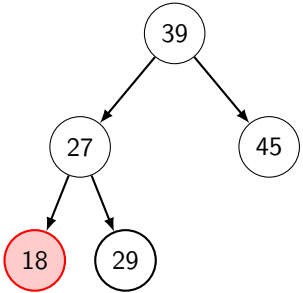
C++ Implementation: BST Deletion I

```
Node* deleteNode(Node* root, int val) {
    if (root == nullptr) return root; // Case: Value not found
    if (val < root->data)
        root->left = deleteNode(root->left, val);
    else if (val > root->data)
        root->right = deleteNode(root->right, val);
    else {
        if (root->left && root->right) {  // Case 3: Two Children
            Node* temp = findLargestNode(root->left);
            root->data = temp->data;  // Replace with predecessor
            root->left = deleteNode(root->left, temp->data);
        }
```

C++ Implementation: BST Deletion II

```
        else {          // Case 1 & 2: Leaf node or One child
            Node* temp = root;
            if (root->left == nullptr)
                root = root->right; // Replace with right child
            else if (root->right == nullptr)
                root = root->left;  // Replace with left child
            delete temp; // Free memory
        }
    }
    return root;
}
```

Visualizing Deletion: Case 1 (Leaf Node)

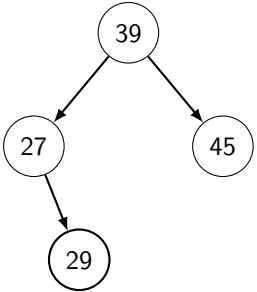


Step 1: Identify Case

The target node (18) is a leaf node (both children are NULL).

```
// val = 18 found
else {          // Case 1: Leaf node or One child
    Node* temp = root;
    if (root->left == nullptr)
        root = root->right; // Replace with right
                child which is nullptr
```

Visualizing Deletion: Case 1 (Leaf Node)

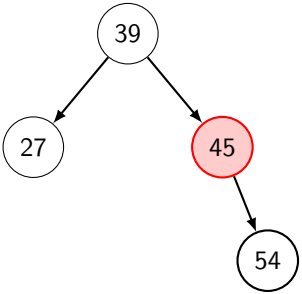


Step 2: Free Memory

The node is deleted and the memory is reclaimed.

```
        delete temp; // Free memory
    }
    return root
}
```

Visualizing Deletion: Case 2 (One Child)

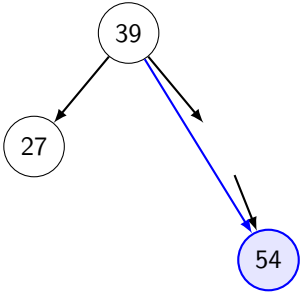


Step 1: Identify Case

The target node (45) is a leaf node (both children are NULL).

```
// val = 45 found
else {          // Case 1: Leaf node or One child
    Node* temp = root;
    if (root->left == nullptr)
        root = root->right; // Replace with right
              child which is nullptr
```

Visualizing Deletion: Case 2 (One Child)



Step 2: Free Memory

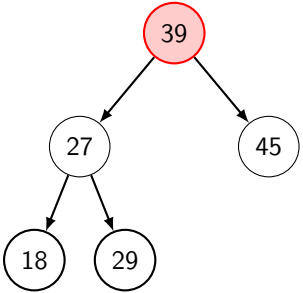
The node is deleted and the memory is reclaimed.

```
        delete temp; // Free memory
    }
    return root
}
```

Question:

We were at node 45 and don't have the access to parent 39. How did the connection from 39 to 54 established?

Visualizing Deletion: Case 3 (Two Children)

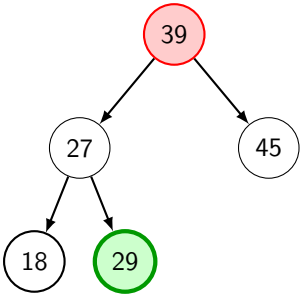


Step 1: Identify Case

The target node (39) has two children. We must find a replacement.

```
// val = 39 found at root
if (root->left && root->right) {
    // Proceed to Case 3 logic...
```

Visualizing Deletion: Case 3 (Two Children)

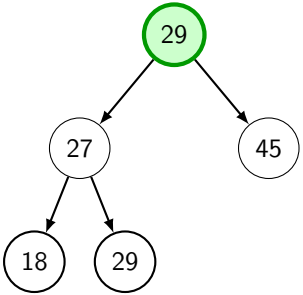


Step 2: Find Predecessor

Find the largest node in the left sub-tree to maintain BST order.

```
Node* temp = findLargestNode(root->left);
// temp now points to node 29
```

Visualizing Deletion: Case 3 (Two Children)

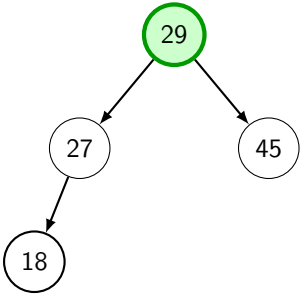


Step 3: Replace Value

Copy the predecessor's data into the target node's position.

```
root->data = temp->data;
// Root value is now 29
```

Visualizing Deletion: Case 3 (Two Children)

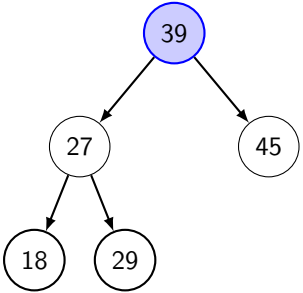


Step 4: Delete Predecessor

Remove the original predecessor node (now a Case 1 or 2 deletion).

```
root->left = deleteNode(root->left, temp->data);
// Node 29 is now removed
```

Utility: Finding the Smallest Element

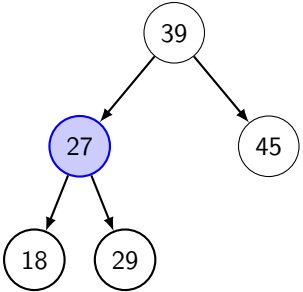


Step 1: Start at Root

Move to the left child if it exists.

```
Node* findSmallest(Node* root) {
    if (root == nullptr ||
        root->left == nullptr)
        return root;
```

Utility: Finding the Smallest Element

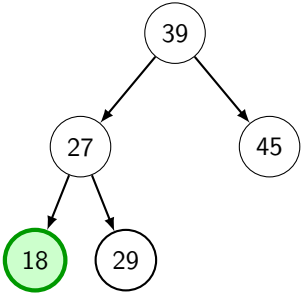


Step 2: Traverse Left

Keep moving to the left until you hit a node with no left child.

```
    return findSmallest(root->left);
}
```

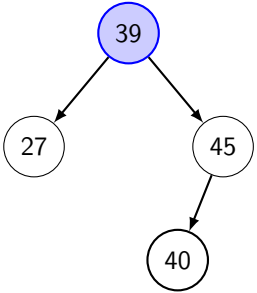
Utility: Finding the Smallest Element



Step 3: Found!

The leftmost node (18) is the smallest element in the BST.

Utility: Finding the Largest Element

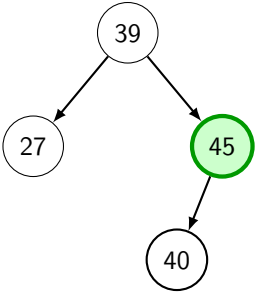


Step 1: Start at Root

Move to the right child if it exists.

```
Node* findLargest(Node* root) {
    if (root == nullptr ||
        root->right == nullptr)
        return root;
```

Utility: Finding the Largest Element

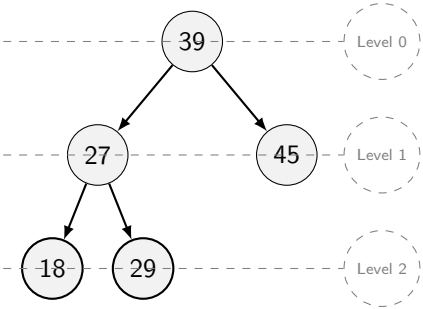


Step 2: Found!

Since there is no right child for node 45, it is the largest element.

```
    return findLargest(root->right);
}
```

Utility: Calculating Tree Height

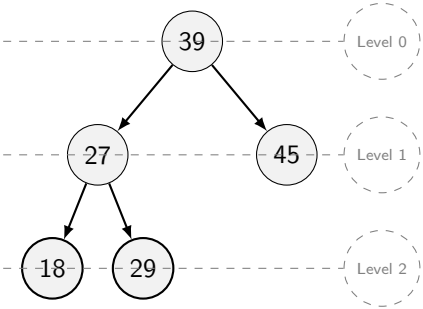


Recursive Logic

To find the height of a node, calculate the height of its sub-trees first.

```
int Height(Node* root) {
    if (root == nullptr)
        return 0;
    else {
        int leftHeight = Height(root->left);
        int rightHeight = Height(root->right);
```

Utility: Calculating Tree Height



Final Calculation

Compare the two heights and add 1 for the current level.

```
        if (leftHeight > rightHeight)
            return (leftHeight + 1);
        else
            return (rightHeight + 1);
    }
}
```

Utility: Counting Nodes I

1. Total Nodes

```
int totalNodes(Node* root) {
    if (root == nullptr) return 0;
    return (totalNodes(root->left) + totalNodes(root->right) + 1);
}
```

2. External (Leaf) Nodes

```
int totalExternalNodes(Node* root) {
    if (root == nullptr) return 0;
    if (root->left == nullptr && root->right == nullptr) return 1;
    return (totalExternalNodes(root->left) + totalExternalNodes(root->right));
}
```

Utility: Counting Nodes II

3. Internal Nodes

```
int totalInternalNodes(Node* root) {
    if ( (root == nullptr) || (root->left == nullptr && root->right == nullptr) )
        return 0;
    return (totalInternalNodes(root->left) + totalInternalNodes(root->right) + 1);
}
```

Time Complexity of BST Operations

The time complexity of most BST operations is proportional to the height of the tree (h).

Operation	Average Case	Worst Case (Skewed)
Searching	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Find Min/Max	$O(\log n)$	$O(n)$
Tree Height	$O(n)$	$O(n)$

- n: Number of nodes in the tree.
- **Height**: In a balanced tree, $h \approx \log_2 n$. In a skewed tree, $h = n$.

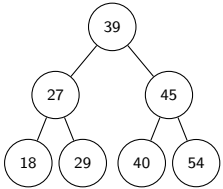
Understanding the Performance Gap

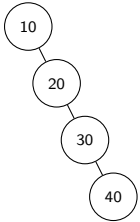
Average Case: $O(\log n)$

In a reasonably balanced tree, each comparison allows us to skip half of the remaining nodes. This logarithmic behavior makes BSTs very efficient for large datasets.

Worst Case: $O(n)$

If elements are inserted in sorted order, the tree becomes **skewed**. It effectively turns into a Linked List, forcing us to visit every node.





Space Complexity Analysis

The space complexity depends on how the algorithm is implemented:

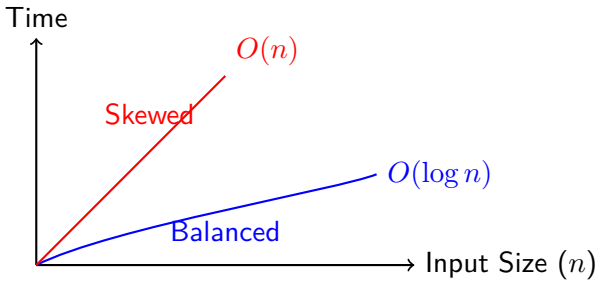
- **Recursive Implementation:**
  - Requires stack space for recursive calls.
  - **Space Complexity:** $O(h)$, where h is the height of the tree.
  - Worst case (skewed tree): $O(n)$.
  - Average case (balanced tree): $O(\log n)$.
- **Iterative Implementation:**
  - Does not use the call stack.
  - **Space Complexity:** $O(1)$ auxiliary space.

Design Choice

While recursion is more intuitive and easier to code for trees, iterative versions are more memory-efficient for very deep trees.

Summary: Why use a BST?

- **Efficiency:** Better than linked lists for searching ($O(\log n)$ vs $O(n)$).
- **Dynamic:** Unlike arrays, BSTs can easily grow and shrink without expensive re-allocations.
- **Sorted Data:** In-order traversal gives data in sorted order for "free".



🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 10**:(Covered topics in the slides only)

Thank You!

👏 Thank you for your time and attention. 👏

🗨 Any questions or clarifications. 💭